

Computer Workshop 3: Linear models with a continuous predictor

Roos & Pinard

Last updated: 2024-09-17

Table of Contents

Workshop Structure	1
1. Formulate a Research Question	2
Assessing the Normal Distribution Assumption.....	2
Important Clarification	3
Question Set One.....	3
2. Perform Exploratory Data Analysis	4
3. Identify Any Hidden Assumptions	5
Question Set Two	5
4. Fit an appropriate model.	5
Mapping the Equation to the Code	6
Parameter Meanings	6
Question Set Three	7
5. Diagnose the model and check assumptions.....	8
6. Summarise the results.....	8
Use an R package to create the figure for you.	9
Create the prediction and figure yourself.....	10
A new challenger	12
Assumption table.....	14

Workshop Structure

As always, we are using the workflow below to structure the workshops:

1. Formulate a research question.
2. Perform exploratory data analysis.
3. Identify any hidden assumptions.
4. Fit an appropriate model [Focus of today].

5. Diagnose the model and check assumptions.
6. Summarise the results.
7. Interpret and provide inferences.

However, starting this week, we'll start by applying this entire workflow to a single dataset for a more integrated experience, before giving you a new dataset to work through independently.

1. Formulate a Research Question

For this first workshop on linear models, we'll use a dataset introduced in a previous workshop to start with something familiar: the `boto.txt` dataset from Workshop 2. Here's a reminder of what's included in the dataset:

- `sex`: The sex of the boto dolphin.
- `drought`: The number of years out of the past 20 where drought was observed in the region where the dolphin was recorded.
- `length`: The length of the boto dolphin (in cm).

For this analysis, we aim to determine whether the number of droughts in the past 20 years affects the length of boto dolphins. The broader context for this question lies in concerns that droughts may become more frequent in parts of the Amazon basin due to climate change and dam construction. In other words, if droughts increase, could boto dolphins become smaller and potentially be in poorer condition?

From this, we know two things: - The response variable is `length`. - The predictor variable (also known as the explanatory variable or covariate) is `drought`.

In essence, we want to understand how `length` changes as `drought` varies.

Before proceeding, it's crucial to consider whether the data-generating process for boto length (i.e., all factors influencing dolphin size) can be reasonably approximated by a *Normal* distribution. Specifically, we're asking:

$$length_i \sim Normal(\mu_i, \sigma)$$

where $length_i$ represents the length of dolphin i , which we assume follows a *Normal* distribution with a mean of μ_i (the model's predicted length for dolphin i) and a standard deviation of σ .

Assessing the Normal Distribution Assumption

To decide whether assuming a *Normal* distribution is reasonable, we start by asking if `length` is continuous or discrete. A discrete variable could *only* take integer (or whole numbers) values like 1 cm, 2 cm, etc. Phrased alternatively, we could never have a dolphin that was 270.1 cm. It would either be 270 or 271 cm. In contrast, while our data are rounded to the nearest centimeter, the actual lengths of the dolphins aren't constrained to such increments; `length` is inherently continuous. A *Normal* distribution is appropriate for generating continuous values (though there are others). To see this visually, you can run the following code which uses `rnorm()`, or "randomly generate n values from a *Normal* distribution":

```
rnorm(n = 10, mean = 200, sd = 10)
```

After running the code, you'll notice that the simulated values look pretty similar to the length of our boto dolphins, suggesting that a *Normal* distribution is a good fit in this respect (note we're only using `rnorm()` here for demonstration purposes, you wouldn't really do this in a formal analysis). The only difference between the `rnorm()` values and the actual length measurements we have is that the researchers seem to have rounded their length measurements.

Next, we should ask if there are any natural bounds on the data and whether our model might predict values near these bounds. For boto length, the natural lower bound is zero — i.e. no adult animal can be 0 cm long. Although this suggests that a *Normal* distribution could theoretically predict impossible (negative or zero) lengths, the minimum observed length in our data is well above zero. Therefore, the risk of the model predicting such extreme values is minimal.

In practice, this makes a *Normal* distribution a reasonably safe choice here. However, we should still check the model's predictions later to ensure it isn't producing values that are physically implausible. If we the model was predicting the impossible, then we'd switch to another distribution using a "Generalised" Linear Model (or GLM). We'll cover GLMs at the end of the course.

Important Clarification

Working through the process of selecting an appropriate distribution might feel overwhelming when you're new to statistics. However, with experience, these decisions become intuitive and can often be made in a matter of seconds.

Note: Deciding which distribution to use is entirely distinct from the assumption of normality in linear models, formally known as the assumption of "normally distributed errors" (or sometimes "normally distributed residuals"). The choice of distribution is a consideration based on the nature of the response variable, and you can make this decision before even examining the dataset.

Question Set One

1. Does a *Normal* distribution generate discrete or continuous values?

A *Normal* distribution generates continuous values. While these values might subsequently be rounded in a dataset (e.g., to the nearest cm), the underlying process is still generating continuous values.

2. Using `rnorm(n = 100, mean = 50, sd = 20)`, what's the smallest value you generate when running it? Try changing the standard deviation (`sd =`) to see what impact this has.

Because this involves randomly generated numbers, your results will vary each time you run the code. However, there's a good chance that some of your generated values will be below zero, especially if you increase the standard deviation.

The point of this question is to emphasize that a *Normal* distribution can produce values below zero. This is important because it means that, in theory,

our model could predict that botos are zero or even negative centimeters long, which is biologically impossible.

```
# You can quickly find the minimum value using the `min()` function:  
# min(rnorm(n = 100, mean = 50, sd = 20))
```

3. If the values generated by `rnorm(n = 100, mean = 50, sd = 20)` were your response variable, would it be reasonable to assume a *Normal* distribution in your analysis?

```
# Yes. We used a Normal distribution to generate the data, so a model that  
# assumes a Normal distribution (like a linear model) is entirely appropriate  
.
```

```
# This question is intended to be straightforward, but I'm also trying to highlight  
# that there is a challenge in the boto example, where we're balancing biology with  
# statistics; Botos cannot be 0 or negative cm long, yet a Normal distribution  
# technically allows for such values, but we can probably get away with using  
# a Normal distribution in this case because none of our observed values are  
# particularly close to zero.
```

4. If your data is generated according to a *Normal* distribution, does that mean you have met the assumption of Normality?

```
# No, generating data according to a  $\text{\$Normal\$}$  distribution does not automatically  
# satisfy the assumption of normality. The assumption of normality specifically  
# refers to the distribution of the residuals (i.e., the differences between the  
# observed and predicted values) in your model – neither the raw data itself nor  
# the distribution we assume generated the data.
```

```
# Even if the underlying data is normally distributed, the residuals might not be,  
# depending on how well our model fits the data. We can only check the residuals  
# after fitting the model to confirm whether they are approximately normally distributed.
```

2. Perform Exploratory Data Analysis

Since you've already worked with this dataset in a previous workshop (this is the same data you used to learn `ggplot2` and data visualisation, which is a key part of getting to know your data set), we can skip the exploratory data analysis (EDA) step in today's workflow. However, if you'd like a refresher, you can either continue using the script from Workshop 2 or copy and paste the relevant EDA code into your new script for today.

3. Identify Any Hidden Assumptions

Before we proceed, take some time to think about the various assumptions we'll be making when using this dataset for modeling. If you need a reminder of what the assumptions are, they're included at the end of this document.

Consider whether we are likely to break any of these assumptions based on your understanding of the data and your exploratory data analysis (EDA).

Question Set Two

1. Which assumptions do you believe are very likely to be violated?

One potential issue could be the assumption of additivity. Last week, we saw that different sexes might be affected differently by drought, suggesting an interaction between sex and drought. In instances where you have "overlap" between two or more variables, the assumption of additivity is likely to be broken.

2. Which assumptions could we check *after* fitting the model?

All of the assumptions related to model residuals. Remember - residuals are the difference between what the model predicts and what we observed. We can't check that until we've run the model. We can also check linearity after (but also with EDA) but there is no way to check the others.

Another concern could be the independence of errors. It's possible that dolphins measured in the same region are related (e.g., larger dolphins might produce larger offspring), which could lead to correlated errors. For instance, if a parent is smaller than predicted by the model, and we don't fully understand why, their offspring may also be smaller than expected for the same unknown reasons. This shared unexplained variation would violate the assumption of independence in our model (i.e. misunderstanding one dolphin should not make us more likely to misunderstand another dolphin).

4. Fit an appropriate model.

Now that we've gone through the preliminary steps, it's time to dive into the actual statistics. We're going to fit a linear model of the form:

$$length_i \sim Normal(\mu_i, \sigma)$$

$$\mu_i = \beta_0 + \beta_1 \times drought_i$$

Where:

- $length_i$ is the length of dolphin i , assumed to follow a *Normal* distribution with a mean of μ_i (what the model predicts for dolphin i) and a standard deviation of σ .
- μ_i represents the linear relationship between $drought_i$ (the number of drought years) and dolphin length, defined by two parameters:

- β_0 : The intercept of our line (how long we expect a dolphin to be when the number of droughts is zero).
- β_1 : The slope (how much *length* is expected to change for each additional year of drought).

If you need a refresher, feel free to go back to the lecture slides or recording.

That's the conceptual model. But how do we translate that into some R code? It's pretty straightforward, and we can also store the model output so we can use it again later on without having to rerun the model each time. Here's how we do it:

```
boto_model <- lm(length ~ drought, data = boto)
```

Mapping the Equation to the Code

Here's how the mathematical model corresponds to the R code:

- `lm()` tells R that we are fitting a linear model, which, by default, means that we assume the response variable follows a *Normal* distribution.
- The formula `length ~ drought` represents the relationship we're modeling. In mathematical terms, this corresponds to $\mu_i = \beta_0 + \beta_1 \times drought_i$. The *length* variable is our response (sometimes called the "dependent") variable, and *drought* is our predictor (or "independent" or "explanatory") variable.

Now, let's get to the interesting part: what does the model tell us? We can use the `summary()` function to see the results and interpret what the model says:

```
summary(boto_model)
```

When we run the `summary()` function on our model, we get a lot of information, but for starters, we are particularly interested in the Estimates section. From this, we can see the estimated values for β_0 and β_1 : 213.281 and -5.308, respectively. But what do these values mean?

Parameter Meanings

When we wrote out our equation above, we said " β_0 : The intercept of our line (how long we expect a dolphin to be when the number of droughts is zero)" and " β_1 : The slope (how much *length* is expected to change for each additional year of drought)" but why is that? How do we know what these parameters means? To show you how you can do this, let's revisit our model equation, where we'll start by figuring out what the intercept is:

$$\mu_i = \beta_0 + \beta_1 \times drought_i$$

$drought_i$ is whatever value *drought* is for row *i* of our dataset. As a thought exercise, imagine a scenario where there has never been a drought, and $drought = 0$. In this case, the equation becomes:

$$\mu_i = \beta_0 + \beta_1 \times 0$$

Because any number multiplied by zero becomes zero, we can simplify this to:

$$\mu_i = \beta_0$$

And we're left with β_0 . This is why β_0 represents the estimated length of boto dolphins when there has been no drought. And according to our model, the intercept value is 213.3 cm. So, if no droughts have occurred, the model predicts that dolphins would be approximately 213.3 cm long. This is why the intercept is often described as “the value of Y when X is equal to zero”.

If β_0 represent the length of botos when drought is zero, then β_1 must be the value that describes how much the length of the dolphins changes with each additional year of drought. From our model summary, β_1 is -5.308. This means that for each additional year of drought, the length of an average dolphins decreases by roughly 5.3 cm.

So if we were to draw a line, we'd start at $drought = 0$ and $length = 213$, then for every additional year (or a “unit increase”) $length$ would reduce by 5.3.



And that's it! All the actual statistics we need has been run and summarised in just two lines of code using `lm()` and `summary()`. What now?

Question Set Three

1. Replace the blank areas of the equation with the values that have now been estimated by the model.

$$length_i \sim Normal(\mu_i, \sigma)$$

$$\mu_i = _ + _ \times drought_i$$

```
# The first blank should be 213.3 and the second blank should be -5.3
# Or mu = 213.3 + -5.3 * drought
```

2. Using your completed equation from 1., what value of μ (or boto length), would you predict if there had been *zero* years of drought?

```
# We can just replace the drought from the above equation with zero:  
# 213.3 + -5.3 * 0 = 213.3
```

3. Using your completed equation from 1., what value of μ (or boto length), would you predict if there had been *eight* years of drought?

```
# We can just replace the drought from the above equation with eight:  
# 213.3 + -5.3 * 8 = 170.9
```

4. What would you predict the difference in length be for a boto that had experienced two years of drought, and another than had experienced seven years of drought?

```
# Start with the boto with 2 years of drought  
# 213.3 + -5.3 * 2 = 202.7
```

```
# Then with the boto with 7 years of drought  
# 213.3 + -5.3 * 7 = 176.2
```

```
# Then just subtract the two:  
# 202.7 - 176.2 = 26.5
```

```
# The boto with 7 years of drought will be 26.5 cm shorter *according to our  
model*
```

5. Diagnose the model and check assumptions.

We'll cover this on Friday.

6. Summarise the results.

We'll come back to "5. Diagnose the model and check assumptions." on Friday. For today we'll go through how to take these results and convert them into a figure.

To do so we'll show you two options.

- Option 1: Use an R package to create the figure for you.
- Option 2: Create the prediction and figure yourself.

Option 1 is nice because it's really quick and easy. The downside, though, is that you don't get much control in what the figure looks like, and these packages can often obscure what's going on, leading to confusion (e.g. "How did you make this figure?". *shrug*).

Option 2, on the other hand, can be more fiddly, but has the benefit of giving you flexibility, and makes the underlying process pretty transparent.

We'll start with Option 1.

Use an R package to create the figure for you.

First, install a package called `ggeffects`:

```
install.packages("ggeffects")
```

As with other R packages, once it's installed on your machine you shouldn't need to install it again. When it's finished loading, make sure to load it:

```
library(ggeffects)
```

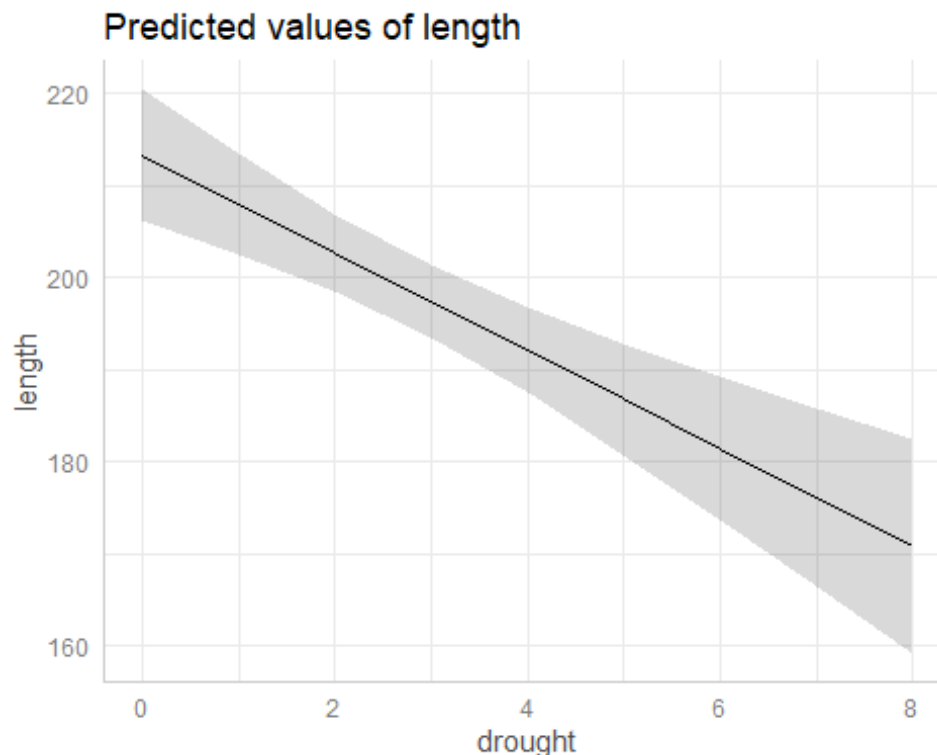
```
## Warning: package 'ggeffects' was built under R version 4.3.3
```

Next, we're going to use a function within `ggeffects` to create predictions from our model which we'll store in a new object called `preds` (or whatever you prefer to call it), which we'll plot subsequently. This is where storing the model output becomes really handy:

```
preds <- ggpredict(boto_model)
```

Now we just need to plot it:

```
plot(preds)
```



And there we have it - with 95% confidence thrown in as a bonus! (We'll cover 95% CI and what they mean next week). As easy as that. Buuuuut.... there are stylistic choices that I, personally and subjectively, don't like. I wanna decide what's going on, not some automated software!

Create the prediction and figure yourself.

It's actually not much harder to make the figure ourselves. We've actually already learnt all of the techniques we need to do so. Let's get to it.

For starters, we're going to make a *fake* dataset. This dataset is going to contain, initially, just one variable; drought, and we're going to keep it as simple as we can. We just want drought values of 0, 1, 2, 3, 4, 5, 6, 7 and 8. We can create this kind of data super easily in R using `0:8`. Try entering `0:8` in the console to see what it does, if you're unsure. To create the fake dataset we use the `data.frame()` function. Bringing this all together, we get:

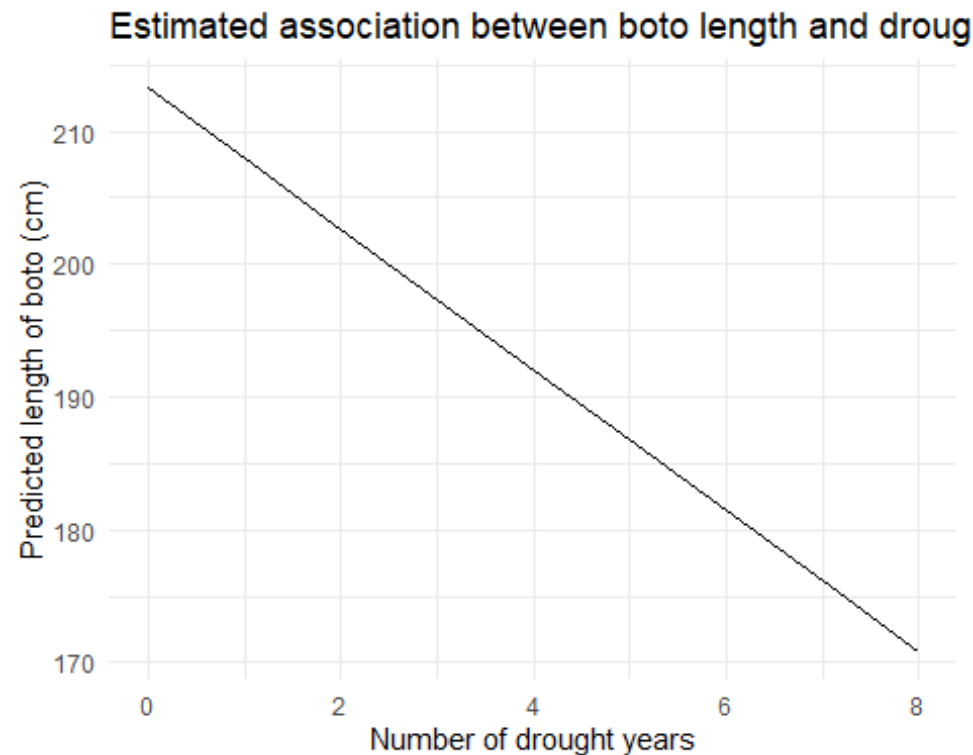
```
fake <- data.frame(  
  drought = 0:8  
)
```

In Question Set Three, you specified values of drought and used your equation to predict boto length by hand. What we're going to do next is *exactly* the same, we're just going to be "lazy" and have R do it for us using the `predict()` function. To be clear, it's doing *exactly* the same thing you did; just taking a value of drought and predicting boto length. As with `ggeffects` we say what the model was, but we're also going to tell `predict()` to use a new dataset to make the predictions; our fake dataset. We're then going to add these predictions into the fake dataset to make plotting easier for us.

```
fake$fit <- predict(boto_model, newdata = fake)
```

Now we have a dataset that contains all values of drought we're interested in, as well as the predicted length (called `fit` in my code above). So we have a dataset (`fake`) and we want to make a plot. You guys know how to do this, using `ggplot2`!

```
ggplot(fake) +  
  geom_line(aes(x = drought, y = fit)) +  
  labs(x = "Number of drought years",  
       y = "Predicted length of boto (cm)",  
       title = "Estimated association between boto length and drought") +  
  theme_minimal()
```



The full process here, from loading a dataset to getting a figure that enables us to see the association between drought and boto length can be done in a surprisingly few lines of code. Here's my how I'd code this:

```
# Deon Roos
# Analysis of boto length in relation to droughts

# Load data
setwd("my/file/path")
boto <- read.table("boto.txt", header = T)

# EDA
summary(boto)
str(boto)
ggplot(boto) + geom_point(aes(x = drought, y = length))
ggplot(boto) + geom_point(aes(x = drought, y = length, colour = sex))
ggplot(boto) + geom_point(aes(x = drought, y = length, colour = sex)) + facet
_wrap(~sex)

# No weird observations detected in EDA

# Concerns prior to modelling
# Measurement of botos is very accurate - unsure how this was measured
# Validity - depending on how measurement was taken
# Independence - seems like that families would be measured
# Additivity - drought may impact sexes differently

# Analysis
```

```

boto_model <- lm(length ~ drought, data = boto)

# Diagnostics
# Covered in next workshop

# Inference
summary(boto_model)

fake <- data.frame(drought = 0:8)
fake$fit <- predict(boto_model, newdata = fake)
ggplot(fake) +
  geom_line(aes(x = drought, y = fit)) +
  labs(x = "Number of drought years",
       y = "Predicted length of boto (cm)",
       title = "Estimated association between boto length and drought") +
  theme_minimal()

```

The entire workflow is done in about 40 lines of code and about a quarter of that is comments. More importantly, the entire analysis of this dataset is done responsibly, with useful (in my mind) comments, and is now saved in a script such that I can share this with anyone who might want to see it. That last point is a big one. As mentioned previously, there are scientists out there who routinely fake data. Being able to provide a script which clearly demonstrates every step you took in a piece of analysis is a huge help in detecting those kinds of issues. Indeed, I'd recommend attaching your R scripts as an appendix when you submit your Honour's project, not because the reviewers will go through it in any amount of detail, but simply to instil some good research practices.

As a tip, don't confuse the length of a script with complexity or the intelligence of the coder. When I did my Honour's project, my R script had something like 1,000 lines of code. It wasn't because I was doing anything particularly complicated (my analysis was more or less equivalent to the model you just ran) but because I didn't really know what I was doing, so I threw everything I could find at it. Often less is more in coding.

A new challenger

Using the skills you've just developed, you're now going to analyse an entirely new dataset, the `beetles.txt` dataset on MyAberdeen. This data comes from a lab experiment from researchers who were interested in understanding how temperature impacts fecundity in burying beetles (*Nicrophorus vespilloides*), ultimately attempting to understanding how short term (i.e. 6 hours) exposures to temperature changes may affect how the species may respond to climate change.

The data contains:

- `offspring` - the number of offspring produced by the beetles
- `temperature` - the temperature at which the beetles were exposed to for 6 hours a day

Using this data, carry out an EDA, consider which assumptions are likely violated, determine an appropriate model and fit it, then plot the results.

You can go about this in any manner you deem appropriate but keep in mind that we may ask you to analyse an equivalent dataset in a Quiz, so make sure you are comfortable with the workflow.

If you don't manage to finish, don't worry, we'll return to these models on Friday.

The code below captures the full workflow as if I were analysing this myself:

Deon Roos

Analysis of burying beetle fecundity

How does temperature associate with fecundity?

Packages -----

library(ggplot2) *# For plotting*

library(ggeffects) *# For quick plotting of model fit*

Data -----

(Not running these here because of how this document is created)

setwd(H:/BI3010/)

beetles <- read.table("beetles.txt", header = TRUE)

EDA -----

str(beetles)

n = 234

summary(beetles)

minimum is 1 offspring, seems a bit low?

ggplot(beetles) +

geom_histogram(**aes**(x = offspring), colour = "white", binwidth = 1) +

theme_minimal()

After plotting, offspring of 1 seems reasonable - just on the low end

ggplot(beetles) +

geom_histogram(**aes**(x = temperature), colour = "white", binwidth = 1) +

theme_minimal()

Fairly even spread of temperatures between -10 and 35 C

ggplot(beetles) +

geom_point(**aes**(x = temperature, y = offspring)) +

theme_minimal()

Looks like a pretty clear non-linear trend

Offspring increases then decreases as temp increases

NOTE

Assumption of linearity very likely broken

Parameter estimates likely biased meaning results of model are likely not suitable for inference

CAUTION when interpreting model results

```

# Hidden assumption -----
# - See above - Linearity almost certainly broken
# - Equal variance is possible? As temp increases maybe offspring begins to fluctuate a lot (plot looks ok though)
# - Representativeness maybe? This is a lab population, so care needed when extrapolating to wild populations

# Model fit -----
beet_mod <- lm(offspring ~ temperature, data = beetles)

# Diagnose -----
# We'll cover this on Friday

# Summarise -----
summary(beet_mod)
# Model predicts 25.3 offspring are produced at 0 degrees (intercept is when X is 0, or temp is 0)
# For every 1 degree change in temperature, we gain an additional 0.5 baby beetles
# BUT(!) this model is biased, and so these parameters are unreliable, so we should not use this model

preds <- ggpredict(beet_mod)
plot(preds, add.data = TRUE)
# The plot above includes the raw data (via add.data = TRUE)
# The linear fit to the non-linear relationship is pretty clear

# Interpret -----
# This model, as is, should not be used to make any kind of inference
# (In reality, we'd need to change the model to allow temperature to have a non-linear effect)
# (But given you don't know how to do that, we should stop here and advise/instruct the results are not interpreted)

```

Assumption table

Assumption	Description	Example of Violation
1. Validity	The data is able to answer the research question effectively.	Using IQ to measure overall intelligence (what does IQ even measure?).
2. Representativeness	The data reflects the population or group being studied.	Using survey data from a rural area to draw conclusions about an urban population.
3. Additivity	Effects of predictors are additive and there are no interactions between them.	Assuming that study hours and sleep hours independently affect exam scores without considering their combined effect (imagine a

Assumption	Description	Example of Violation
		Venn diagram with study and sleep overlapping).
4. Linearity	The relationship between predictors and the outcome is linear.	Assuming plant growth will continue to increase indefinitely without ever levelling out.
5. Independence of Errors	Observations are independent of each other. (Related to residual errors.)	If modelling height, the data come from the same group of children over time.
6. Equal Variance of Errors	Residuals have constant variance across all levels of predictors ("homoscedasticity"). (Related to residual errors.)	If modelling income, variability in salaries is not the same across all levels of a company's hierarchy.
7. Normality of Errors	Residuals should be normally distributed. (Related to residual errors.)	If modelling income, the presence of any extremely wealthy individuals will be hard for the model to understand and explain.